

INSTITUTO FEDERAL DA PARAÍBA
CAMPUS CAMPINA GRANDE
BACHARELADO EM ENGENHARIA DA COMPUTAÇÃO
DISCIPLINA DE ED e LAB. ED
PROF. VICTOR ANDRÉ PINHO DE OLIVEIRA

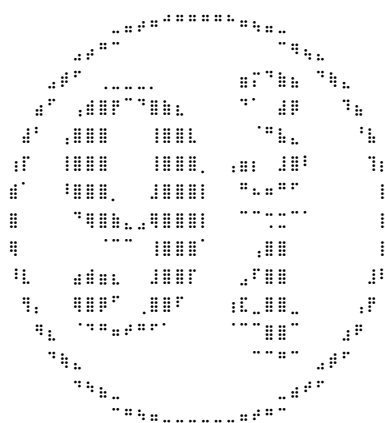
Atividade Unidade I - 3 - Fundamentos C

Ponteiros e Funções

Instruções

Responda às questões abaixo. Pode usar este próprio documento. Questões práticas devem ser anexadas separadamente.

Pegue o ~~Guanabara~~ Expresso de Hogwarts e vem...



Legenda:

Quirrell: Questões mais simples e diretas

Olho-Tonto: Questões de nível intermediário

Dolores: Questões de nível mais difícil, exigentes

Voldemort: Questões com nível de exigência altíssimo, desafiadoras

Questões

Teóricas

1. **Quirrell** O que é um ponteiro? Existe alguma diferença entre uma variável e um ponteiro?

R: Um ponteiro é uma variável que guarda o endereço de determinada região de memória, a diferença é justamente essa uma variável comum guarda um valor direto, (como um número ou texto), enquanto um ponteiro guarda é uma variável que guarda o endereço de memória onde outro dado está localizado.

2. **Quirrell** Como devemos inicializar um ponteiro de forma segura quando ainda não sabemos ainda para onde ele vai apontar?

R: Para evitarmos apontamentos estranhos, que podem ser completamente aleatório, devemos inicializar um ponteiro com a constante NULL, é uma forma segura, como se ele apontasse para o Terra.

3. **Quirrell** O que acontece quando não inicializamos um ponteiro e tentamos desreferenciá-lo?

R: Ninguém sabe, ocorre um comportamento indefinido, o ponteiro vai conter lixo de memória, um endereço aleatório que já estava lá, o programa pode até crashar se apontar para uma região restrita pelo sistema operacional.

4. **Olho-Tonto** O que faz o seguinte trecho de código?

```
int a = 100;
int b = 200;
int *p = &a;

*p = 0;

p = &b;

*p = 0;
```

R: Muda a o valor de A e B para 0.

5. **Quirrell** O que é uma função?

R: É um bloco de código que realiza determinada tarefa e são reutilizáveis, elas servem para dividir o programa em partes em menores e organizadas, evitando repetição de código. É formada por: tipo de retorno, nome, parâmetros e corpo.

6. **Quirrell** O que significa dizer que só existe passagem argumentos por valor em C?

R: Que quando passamos uma variável para uma função, o programa cria uma cópia exata desse valor em uma nova área de memória, e não a variável original.

7. **Olho-Tonto** De que maneira a linguagem C nos permite alterar o valor de uma variável passada para uma função? Explique.

R: Passando o endereço da variável para a função, um ponteiro para a variável, e depois desreferenciando na função para acessar o conteúdo da variável.

8. **Olho-Tonto** Escreva uma função chamada **max** que recebe dois argumentos e retorna o maior deles.
9. **Olho-Tonto** Escreva uma função chamada **primo** que recebe um número e retorna 1 para primo e 0 para não primo.
10. **Olho-Tonto** Escreva uma função que recebe um array A, o tamanho do array (T) e um número N inteiro. A função deve retornar a soma de $A[i] * N$, onde i representa os índices dos elementos do array.
11. **Olho-Tonto** Escreva uma função chamada **reset** que recebe um ponteiro para inteiro e atribui zero à variável apontada.
12. **Dolores** Escreva uma função chamada **swap_str** que recebe duas strings e faz o swap entre elas. Suponha que os tamanhos das strings sejam suficientes e corretos para a operação.
13. **Dolores** Usando funções, ponteiros e aritmética de ponteiros, escreva uma função chamada **str_cpy** que aceita duas strings e copia a segunda na primeira, sobrescrevendo-a. Suponha que os tamanhos das strings sejam suficientes e corretos para a operação.
14. **Dolores** Escreva uma função recursiva que recebe um array e retorna a soma de seus elementos.

Voldemort está tramando algo para a próxima lista...

```

      _ _ _ _ _
     / ' - .      - . , - - - ,
    /  \ - ) _ _ _ ' - .      \      - , , - _ \ )
   / _ _ ' - .      \      ' - ,      _ _ _ ( c - ( 6 ) = ( 6 )
  ,  \ ' .      \ - .      ' - ,      > \      " )
 : : : , ' - .      ' - - - ' ( ( " / \ . - = ' /
 : : : : : , ' . _ _ _      , - . ) ' - ' - - '
 ' : : : : : ' - .      / ' . _ |      Y /      - / ' \
      ' ' ' " . _ F      |      - / _ . ' - .      \
          L      \      \ /      ' . _      \

```

.-,,-,| \. \'----, _ _|
// 'L / \, ("--',≡`)7
| \.- : _ , \ /' \'-_L,-' _-
'--' '-._ _/ _L .\'
[(/
) `{
 _ _)